Feuille de route d'amélioration de l'agent Amira

Nov 3, 2026 · @Marouane

Quatorze améliorations, classées par priorité. Chacune se teste avant de passer à la suivante, et la phase 1 doit être terminée avant l'événement.

Vue d'ensemble

Les cinq premiers points évitent un incident visible du public. Les suivants rendent l'agent plus fiable, puis plus agréable.

#	Amélioration	Phase	Effort	Fichiers touchés
1	Verrouiller la configuration de la démo	1 · Avant l'événement	30 min	`.env.local`, `agent.py`
2	Resserrer le déclencheur et ajouter des tests	1 · Avant l'événement	2 h	`agent.py`, `tests/`
3	Page opérateur : démarrer, arrêter, passer aux questions	1 · Avant l'événement	1 jour	`agent.py`, `token_server.py`, nouvelle page
4	Arrêt automatique quand la salle est vide	1 · Avant l'événement	2 h	`agent.py`
5	Limiter la longueur des réponses	1 · Avant l'événement	30 min	`agent.py`
6	Basculer en Q&R seulement en fin normale	2 · Robustesse	30 min	`agent.py`
7	Détecter le départ de l'avatar	2 · Robustesse	1 h	`agent.py`, page opérateur
8	Restreindre les droits des jetons	2 · Robustesse	1 h	`token_server.py`
9	Régénérer les clés LiveKit	2 · Robustesse	15 min	`.env.local`

#	Amélioration	Phase	Effort	Fichiers touchés
10	Figer la version de `livekit-client`	2 · Robustesse	15 min	`frontend/index.html`
11	Donner le script de la conférence aux Q&R	3 · Qualité	1 h	`agent.py`
12	Attendre l'avatar et mesurer sa latence	3 · Qualité	1 h	`agent.py`
13	Moderniser l'outillage	3 · Qualité	1 h	projet, `livekit.toml`
14	Comparer d'autres avatars ou une vidéo pré-générée	4 · Option	1 à 2 jours	`agent.py`

Phase 1 — Indispensable avant l'événement

1. Verrouiller la configuration de la démo

Pourquoi. Trois écarts entre le code et ce qui a été validé peuvent gâcher la séance : le script court chargé par défaut, l'audio à 24 kHz qui dégradait la voix, et deux faits non vérifiés dans `QA_PROMPT`.

À faire.

- Mettre `conference_amira.md` comme script par défaut ; le script court ne se charge que si `CONFERENCE_SCRIPT` le demande.
- Remettre `AZURE_TTS_SAMPLE_RATE` à 48000 par défaut.
- Retirer de `QA_PROMPT` les « modules d'éthique obligatoires » ; vérifier le projet HERACLES (Sfax) ou le retirer.
- Afficher au démarrage, en `WARNING`, le script chargé, sa durée estimée, la voix et la fréquence audio, pour les voir d'un coup d'œil.

Comment tester.

1. Supprimer `CONFERENCE_SCRIPT` de `.env.local` et lancer l'agent : le log doit annoncer `conference_amira.md` et environ 16 minutes.
2. Remettre `CONFERENCE_SCRIPT=conference_amira_court.md` : le log doit annoncer le script court.
3. En mode questions, demander « Les modules d'éthique sont-ils obligatoires ? » : Amira ne doit pas répondre oui.

Réussi si les trois réglages sont visibles au démarrage et qu'aucun fait non vérifié ne sort pendant les questions.

2. Resserrer le déclencheur et ajouter des tests

Pourquoi. « Bonjour à vous, Amira » lance la conférence, et « Après cette introduction, à vous la parole » ne la lance pas.

À faire.

- Retirer les motifs « à vous Amira », « Amira à vous » et « on vous écoute » seul ; garder « nous vous écoutons, Amira ».
- Ne chercher les marqueurs de futur que dans les 3 mots autour de « parole » ou « écoute », et retirer « après » et « ensuite » de la liste globale.
- Écrire `tests/test_trigger.py` avec deux listes de phrases : celles qui doivent déclencher, celles qui ne doivent pas.

Comment tester.

1. Lancer `uv run pytest tests/test_trigger.py` : tous les tests passent.
2. La liste doit contenir au moins : « Amira, à vous la parole », « Amira avoue la parole », « Après cette introduction, Amira, à vous la parole » (oui) ; « Bonjour à vous, Amira », « Je vous donnerai la parole dans un instant », « Nous accueillons Amira » (non).
3. En salle, l'animateur lit sa vraie présentation au micro, puis la phrase de passage : la conférence part seulement à la fin.

Réussi si zéro faux départ et zéro silence sur dix essais de la présentation réelle.

3. Page opérateur

Pourquoi. Si le STT rate la phrase, ou si la conférence part trop tôt, personne ne peut intervenir aujourd'hui.

À faire.

- Dans `agent.py`, ajouter les RPC `amira.stop` (coupe la conférence et revient en attente) et `amira.skip_to_qa` (passe aux questions), à côté de `amira.start_conference`.
- Ajouter un RPC `amira.status` qui renvoie l'état (attente, conférence et numéro de paragraphe, questions).
- Créer une page `operateur.html`, protégée par un code simple, avec les trois boutons et l'état rafraîchi toutes les deux secondes.

Comment tester.

1. Sans parler, cliquer « Démarrer » : la conférence commence.

2. Au paragraphe 3, cliquer « Arrêter » : Amira se tait à la fin de la phrase en cours et repasse en attente ; « à vous la parole » relance alors la conférence depuis le début.
3. Cliquer « Passer aux questions » pendant la conférence : Amira répond ensuite à une question.
4. Double-cliquer « Démarrer » : une seule conférence se lance.

Réussi si chaque bouton agit en moins de trois secondes et que l'état affiché est toujours juste.

4. Arrêt automatique quand la salle est vide

Pourquoi. Avec `close_on_disconnect=False`, Amira et l'avatar restent dans la salle après le départ des humains : bitHuman continue de facturer, et il faut un `lk room delete` manuel.

À faire. Écouter les arrivées et départs de participants. Quand il ne reste plus aucun participant humain depuis 3 minutes (réglable par `EMPTY_ROOM_TIMEOUT_S`), journaliser l'arrêt puis fermer la session. Un humain qui revient avant la fin du délai annule l'arrêt.

Comment tester.

1. Mettre `EMPTY_ROOM_TIMEOUT_S=30` pour le test.
2. Ouvrir la page, attendre l'avatar, fermer l'onglet : 30 secondes plus tard, le log indique l'arrêt, et `lk room participants list ma-salle-test` ne montre plus Amira.
3. Rouvrir la page dans les 30 secondes : rien ne s'arrête.
4. Rouvrir la page après l'arrêt : une nouvelle Amira arrive, sans `lk room delete`.

Réussi si plus aucune session fantôme n'apparaît dans le dashboard LiveKit après une séance.

5. Limiter la longueur des réponses

Pourquoi. Le prompt seul ne suffit pas à empêcher une réponse de deux minutes, et le raisonnement du modèle ajoute de la latence.

À faire. Dans `build_llm()`, régler une limite de tokens de sortie (environ 300, raisonnement compris) et un effort de raisonnement faible, tous deux paramétrables dans `.env.local`.

Comment tester.

1. Poser dix questions, dont trois ouvertes (« Parlez-nous de l'avenir de l'IA »).
2. Chronométrer : chaque réponse doit durer moins de 30 secondes et commencer moins de 3 secondes après la fin de la question.

3. Vérifier qu'aucune réponse n'est coupée au milieu d'une phrase ; si c'est le cas, augmenter la limite.

Réussi si dix réponses sur dix sont complètes, courtes et rapides.

Phase 2 — Robustesse

6. Basculer en Q&R seulement en fin normale

Pourquoi. Le bloc `finally` de `_run_conference` lance `update_agent(AmiraQA())` même quand la tâche est annulée, par exemple à la fermeture de la session ou après le bouton « Arrêter ». Cela peut produire une erreur ou un passage aux questions non voulu.

À faire. Intercepter l'annulation (`asyncio.CancelledError`), la journaliser et la relancer sans basculer. Ne basculer qu'après le dernier paragraphe, ou sur `amira.skip_to_qa`.

Comment tester.

1. Lancer la conférence, puis faire Ctrl+C sur l'agent : aucune trace d'erreur liée à `update_agent` dans la console.
2. Lancer la conférence, puis « Arrêter » : Amira revient en attente, pas en questions.
3. Laisser le script court aller au bout : Amira passe bien en questions.

Réussi si les trois scénarios donnent le bon état, sans erreur.

7. Détecter le départ de l'avatar

Pourquoi. Si le participant `bithuman-avatar-agent` quitte la salle, Amira devient muette et personne ne le sait.

À faire. Surveiller le départ de ce participant, écrire une erreur dans les logs, l'afficher dans la page opérateur, et tenter une relance de la session avatar une fois.

Comment tester.

1. Pendant une session, retirer l'avatar avec `lk room remove-participant ma-salle-test bithuman-avatar-agent`.
2. Vérifier que le log et la page opérateur signalent le départ dans les cinq secondes.
3. Vérifier la relance : l'avatar revient à l'écran, ou un message clair indique l'échec.

Réussi si l'opérateur est toujours prévenu, et l'avatar revient dans la majorité des cas.

8. Restreindre les droits des jetons

Pourquoi. Toute personne qui ouvre la page peut publier son micro, donc parler à Amira et lancer la conférence.

À faire. Deux types de jetons dans `token_server.py` : « salle », qui peut publier le micro, et « spectateur », qui ne fait que recevoir. La page de la salle et la page opérateur demandent le premier avec un code ; toute autre page reçoit le second.

Comment tester.

1. Ouvrir la page sans code, cliquer sur le micro : la publication est refusée, avec un message clair.
2. Ouvrir la page de la salle avec le code : le micro fonctionne et Amira entend.
3. Ouvrir deux pages spectateur en même temps : les deux voient et entendent Amira, et une seule Amira est présente.

Réussi si seule la page autorisée peut parler à Amira.

9. Régénérer les clés LiveKit

Pourquoi. D'anciennes clés sont dans l'historique git ; une clé qui a fui doit être considérée comme compromise.

À faire. Créer une nouvelle clé dans le dashboard LiveKit, mettre à jour `.env.local` (`lk app env -w`), puis supprimer les anciennes clés. Faire de même pour Azure, Groq et bitHuman si leurs clés ont aussi été commitées.

Comment tester.

1. L'agent démarre et affiche `registered worker` avec les nouvelles clés.
2. Une commande `lk` utilisant l'ancienne clé renvoie une erreur 401.
3. `git log -p | grep API` ne montre plus de clé encore valide.

Réussi si aucune clé de l'historique ne fonctionne encore.

10. Figer la version de `livekit-client`

Pourquoi. La page charge la dernière version depuis un CDN ; une nouvelle version majeure peut la casser sans prévenir, y compris le jour J.

À faire. Remplacer l'URL du CDN par une version précise, celle qui fonctionne aujourd'hui (visible dans l'onglet Réseau du navigateur).

Comment tester.

1. Ouvrir l'outil de développement (F12), onglet Réseau : le fichier chargé porte le numéro de version fixé.
2. Refaire un test complet : connexion, vidéo, son, micro.

Réussi si la page fonctionne avec la version fixée, et elle seule.

Phase 3 — Qualité

11. Donner le script de la conférence aux Q&R

Pourquoi. `AmiraQA` ne sait pas ce qu'elle vient de dire : elle répond mal à « Revenez sur l'exemple du rapport de stage », et la base de connaissances doit être tenue à jour à part.

À faire. Ajouter au `QA_PROMPT` le texte du script chargé, sans les marqueurs `@ton` ni `[pause]`, sous un titre « Ce que tu viens de dire dans ta conférence ». Garder la consigne de ne citer aucun autre fait précis. Le script devient la source unique des faits.

Comment tester.

1. Après le script court, demander « Vous avez parlé d'un amphithéâtre, pouvez-vous développer ? » : la réponse reprend l'image de la conférence.
2. Demander un fait absent (« Combien d'étudiants utilisent l'IA en Tunisie ? ») : Amira dit ne pas avoir ce chiffre, sans en inventer.
3. Vérifier que la première réponse ne met pas plus d'une seconde de plus qu'avant.

Réussi si Amira cite sa conférence avec justesse et n'invente toujours rien.

12. Attendre l'avatar et mesurer sa latence

Pourquoi. La documentation LiveKit recommande d'attendre que l'avatar soit dans la salle avant de faire parler l'agent, et fournit des métriques pour savoir d'où viennent les lenteurs.

À faire. Appeler `wait_for_join()` sur la session avatar avant `session.start()`, avec un délai de 30 secondes et un log d'erreur en cas d'échec. Journaliser les métriques `AvatarMetrics` : délai d'arrivée de l'avatar et latence de lecture.

Comment tester.

1. Lancer une session : le log affiche le délai d'arrivée de l'avatar.
2. Dire « Bonjour Amira » dès l'apparition de la vidéo : la réponse est vue et entendue en même temps, sans début de phrase manqué.
3. Noter la latence de lecture sur cinq réponses, pour avoir une référence avant tout changement de fournisseur.

Réussi si aucune parole n'est perdue au démarrage et si les métriques sont enregistrées.

13. Moderniser l'outillage

Pourquoi. Le mode `dev` et `agent_name` dans le code sont dépréciés, le rechargement automatique a disparu, et le projet sur `/mnt/c` ralentit WSL et bloque la boucle d'événements au démarrage.

À faire.

- Déplacer le projet dans le système de fichiers Linux (`~/projets/Avatar-Conversationnel`).
- Créer `livekit.toml` avec `[agent] name = "customerSupport"` , retirer `agent_name` du code, et lancer avec `lk agent dev` .
- Remettre le README à jour d'après `DESCRIPTION_PROJET.md` , et y ajouter une section « Configuration de la démo finale ».

Comment tester.

1. `lk agent dev` démarre l'agent sans avertissement de dépréciation.
2. Modifier une phrase du script court et enregistrer : l'agent recharge seul.
3. Comparer les logs `event loop blocked` au démarrage, avant et après le déplacement : ils diminuent ou disparaissent.
4. Une personne qui ne connaît pas le projet lance la démo en suivant seulement le README.

Réussi si le démarrage est propre, rapide, et reproductible par quelqu'un d'autre.

Phase 4 — Options

14. Comparer d'autres avatars ou une vidéo pré-générée

Pourquoi. La synchronisation actuelle est bonne, mais bitHuman est jugé moyen en réalisme et en latence, il est limité à 720p, et il tourne aux États-Unis. À ne lancer que si la qualité visuelle est jugée insuffisante sur grand écran.

À faire.

- Rendre le fournisseur d'avatar paramétrable (`AVATAR_PROVIDER=bithuman|liveavatar|anam`) : seule la création de l' `AvatarSession` change.
- Tester HeyGen LiveAvatar et Anam sur le script court.
- Option la plus sûre pour la conférence : une vidéo pré-générée du même avatar, l'avatar en direct ne servant qu'aux questions.

Comment tester.

1. Passer le même script court par chaque fournisseur, sur le même PC et le même réseau.
2. Faire noter à l'aveugle, par trois personnes et sur grand écran : naturel du visage, synchronisation des lèvres, fluidité.

3. Comparer la latence de lecture relevée au point 12.
4. Vérifier la limite de durée de session de chaque offre : conférence et questions doivent y tenir.

Réussi si un fournisseur l'emporte nettement à l'aveugle sans dégrader la latence ; sinon, garder bitHuman.

Règles de test

- **Un point à la fois.** Un commit par point, avec son numéro dans le message ; en cas de régression, on sait lequel annuler.
- **Le script court pour les essais, le complet pour valider.** Les tests de chaque point se font avec `conference_amira_court.md` ; la fin de chaque phase se valide avec le script complet.
- **Les tests du déclencheur à chaque modification de `agent.py`** : `uv run pytest` doit rester vert.
- **Noter les résultats** dans un tableau simple : point, date, résultat, remarque.
- **Répétition générale à la fin de la phase 1**, dans la vraie salle, avec le vrai micro et le scénario des organisateurs : conférence complète, dix questions, test de chaque bouton opérateur.

- [ ] Phase 1 terminée et validée en répétition générale
- [ ] Phase 2 terminée
- [ ] Phase 3 terminée
- [ ] Phase 4 décidée (faite ou écartée)